

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.12.10 星期三 11-12 节

实验题目：哲学家就餐问题

一、实验内容

编写一个 c/c++ 程序，实现 5 个哲学家就餐问题。

二、实现步骤

1、环境准备

安装/更新核心编译工具

```
sudo apt-get install -y build-essential
```

2、实现细节

(1) 日志内容

场景	内容
思考	哲学家i: 开始思考，预计耗时time秒
	哲学家i: 思考结束，准备就餐
就餐	哲学家i: 开始就餐（第k次），预计耗时time秒
	哲学家i: 就餐结束
申请就餐	哲学家i: 进入临界区（申请叉子）
	哲学家i: 总就餐次数已达到N次，停止就餐
	哲学家i: 处于饥饿状态，等待左右叉子
	哲学家i: 离开临界区（等待申请结果）
就餐结束	哲学家i: 进入临界区（释放叉子）
	哲学家i: 就餐结束，释放左右叉子（总就餐次数: n/N）
	哲学家i: 离开临界区（叉子已释放）
线程管理	创建哲学家线程i
	哲学家i: 线程启动
	哲学家i: 线程退出（个人就餐k次）

(2) 同步机制

5 个哲学家共享 5 把叉子，每个哲学家需要左右两把叉子才能就餐；

每个哲学家拿叉子前，必须通过 `check()` 函数检查邻居状态，只有左右邻居都不就餐时，才能获取叉子；

用互斥信号量 `mutex` 保护 `state` 数组，确保多个线程不会同时修改状态，避免“虚假唤醒”；

信号量 `forks` 初始值为 0，只有 `check()` 函数判定可以就餐时，才会通过 `sem_post()` 激活，确保不会出现“所有哲学家都持有左叉子等待右叉子”的死锁场景。

3、代码编写（注释详细）

(1) 头文件引入与全局变量

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <stdlib.h>
5  #include <time.h>
6  #include <unistd.h>
7
8  // ===== 全局变量 =====
9
10 // 参数配置
11 #define N 5                // 哲学家数量
12 #define EAT_TIMES 10       // 就餐10次后结束
13 #define THINK_MIN 3       // 思考最短时间（秒）
14 #define THINK_MAX 8       // 思考最长时间（秒）
15 #define EAT_MIN 2         // 就餐最短时间（秒）
16 #define EAT_MAX 10        // 就餐最长时间（秒）
17
18 // 哲学家状态
19 #define THINKING 0        // 思考中
20 #define HUNGRY 1          // 饥饿中
21 #define EATING 2         // 就餐中
22
23 // 全局变量
24 int state[N];             // 存储每个哲学家的当前状态
25 sem_t mutex;              // 互斥信号量
26 sem_t forks[N];          // 每个哲学家对应的信号量
27 int eat_count[N] = {0};   // 每个哲学家的就餐次数
28 int total_eat_count = 0;  // 所有哲学家的总就餐次数
```

(2) 工具函数：获取格式化时间

```
32 // 获取格式化时间
33 void get_current_time(char* time_str, int len) {
34     time_t now = time(NULL);
35     strftime(time_str, len, "%Y-%m-%d %H:%M:%S", localtime(&now));
36 }
```

(3) 工具函数：模拟思考

```
38 // 思考
39 void think(int phil_id) {
40     char time_str[32];
41
42     // 随机思考时间
43     int think_time = THINK_MIN + rand() % (THINK_MAX - THINK_MIN);
44     get_current_time(time_str, sizeof(time_str));
45     printf("[%s] 哲学家%d: 开始思考, 预计耗时%d秒\n", time_str, phil_id + 1, think_time);
46
47     sleep(think_time);
48
49     get_current_time(time_str, sizeof(time_str));
50     printf("[%s] 哲学家%d: 思考结束, 准备就餐\n", time_str, phil_id + 1);
51 }
```

(4) 工具函数：模拟就餐

```
53 // 就餐
54 void eat(int phil_id) {
55     char time_str[32];
56
57     // 随机就餐时间
58     int eat_time = EAT_MIN + rand() % (EAT_MAX - EAT_MIN);
59     get_current_time(time_str, sizeof(time_str));
60     printf("[%s] 哲学家%d: 开始就餐(第%d次), 预计耗时%d秒\n",
61           time_str, phil_id + 1, eat_count[phil_id] + 1, eat_time);
62
63     sleep(eat_time);
64
65     get_current_time(time_str, sizeof(time_str));
66     printf("[%s] 哲学家%d: 就餐结束\n", time_str, phil_id + 1);
67
68     eat_count[phil_id]++; // 就餐次数+1
69 }
```

(5) 工具函数：检查是否能就餐

```
71 // 检查哲学家是否能就餐
72 void check(int phil_id) {
73     // 就餐条件
74     // 1. 当前哲学家处于饥饿状态
75     // 2. 左边哲学家不是就餐状态
76     // 3. 右边哲学家不是就餐状态
77     if (state[phil_id] == HUNGRY &&
78         state[(phil_id + N - 1) % N] != EATING && // 左边哲学家
79         state[(phil_id + 1) % N] != EATING) {      // 右边哲学家
80
81         // 修改状态为就餐中
82         state[phil_id] = EATING;
83
84         // 发送信号量, 允许当前哲学家就餐(唤醒等待的线程)
85         sem_post(&forks[phil_id]);
86     }
87 }
```

(6) 工具函数：拿叉子（申请就餐）

```
89 // 拿叉子（申请就餐权限）
90 // 返回值：1表示可以继续就餐，0表示已达到总就餐次数限制
91 int take_forks(int phil_id) {
92     char time_str[32];
93
94     // 1. 申请互斥锁，保护state数组
95     sem_wait(&mutex);
96     get_current_time(time_str, sizeof(time_str));
97     printf("[%s] 哲学家%d: 进入临界区<申请叉子>\n", time_str, phil_id + 1);
98
99     // 检查是否已达到总就餐次数限制
100     if (total_eat_count >= EAT_TIMES) {
101         printf("[%s] 哲学家%d: 总就餐次数已达到%d次, 停止就餐\n", time_str, phil_id + 1, EAT_TIMES);
102         sem_post(&mutex);
103         return 0; // 不再就餐
104     }
105     total_eat_count++; // 总就餐次数预+1
106
107     // 2. 修改状态为饥饿
108     state[phil_id] = HUNGRY;
109     printf("[%s] 哲学家%d: 处于饥饿状态, 等待左右叉子\n", time_str, phil_id + 1);
110
111     // 3. 检查是否能就餐
112     check(phil_id);
113
114     // 4. 释放互斥锁
115     sem_post(&mutex);
116     get_current_time(time_str, sizeof(time_str));
117     printf("[%s] 哲学家%d: 离开临界区<等待申请结果>\n", time_str, phil_id + 1);
118
119     // 5. 等待就餐权限
120     sem_wait(&forks[phil_id]);
121
122     return 1; // 可以继续就餐
123 }
```

(7) 工具函数：放叉子（结束就餐）

```
125 // 放叉子（释放就餐权限）
126 void put_forks(int phil_id) {
127     char time_str[32];
128     get_current_time(time_str, sizeof(time_str));
129
130     // 1. 申请互斥锁，保护state数组
131     sem_wait(&mutex);
132     printf("[%s] 哲学家%d: 进入临界区<释放叉子>\n", time_str, phil_id + 1);
133
134     // 2. 修改状态为思考中（就餐结束）
135     state[phil_id] = THINKING;
136     printf("[%s] 哲学家%d: 就餐结束, 释放左右叉子<总就餐次数: %d/%d>\n",
137           time_str, phil_id + 1, total_eat_count, EAT_TIMES);
138
139     // 3. 检查左边和右边的哲学家是否能就餐
140     check((phil_id + N - 1) % N);
141     check((phil_id + 1) % N);
142
143     // 4. 释放互斥锁
144     sem_post(&mutex);
145     get_current_time(time_str, sizeof(time_str));
146     printf("[%s] 哲学家%d: 离开临界区<叉子已释放>\n", time_str, phil_id + 1);
147 }
```

(8) 哲学家线程函数

```
151 // 哲学家线程函数
152 void* philosopher_thread(void* arg) {
153     // 解析参数 (哲学家ID)
154     int phil_id = *(int*)arg;
155     char time_str[32];
156
157     get_current_time(time_str, sizeof(time_str));
158     printf("[%s] 哲学家%d: 线程启动\n", time_str, phil_id + 1);
159
160     // 循环就餐直到总次数达到限制
161     while (1) {
162         think(phil_id);           // 思考
163
164         // 尝试拿叉子, 如果返回0说明已达到总就餐次数
165         if (!take_forks(phil_id)) {
166             break; // 退出循环
167         }
168
169         eat(phil_id);             // 就餐
170         put_forks(phil_id);       // 放叉子
171     }
172
173     get_current_time(time_str, sizeof(time_str));
174     printf("[%s] 哲学家%d: 线程退出 %d个人就餐%d次\n", time_str, phil_id + 1, eat_count[phil_id]);
175
176     pthread_exit(NULL);
177 }
```

(9) 主函数

```
181 // 主函数
182 int main() {
183     pthread_t phil_tids[N]; // 线程ID
184     int phil_ids[N];        // 哲学家ID
185     char time_str[32];
186
187     // 1. 初始化
188     srand((unsigned int)time(NULL));
189     time_t start_time = time(NULL);
190
191     // 互斥信号量初始值1, 保证同一时间只有一个线程修改state数组
192     if (sem_init(&mutex, 0, 1) == -1) {
193         perror("sem_init mutex failed"); // 错误提示
194         exit(1); // 终止程序
195     }
196
197     // 每个哲学家的信号量forks初始值0 (默认不可就餐, 需通过check()激活)
198     for (int i = 0; i < N; i++) {
199         if (sem_init(&forks[i], 0, 0) == -1) {
200             perror("sem_init forks failed");
201             exit(1);
202         }
203         state[i] = THINKING; // 初始状态: 所有哲学家都在思考
204     }
205
206     printf("=====\n");
207     printf("哲学家就餐实验: 哲学家%d个, 共就餐%d次\n", N, EAT_TIMES);
208     printf("=====\n");
209
210     // 2. 创建哲学家线程
211     for (int i = 0; i < N; i++) {
212         phil_ids[i] = i;
213         if (pthread_create(&phil_tids[i], NULL, philosopher_thread, &phil_ids[i]) != 0) {
214             perror("pthread_create failed");
215             exit(1);
216         }
217         get_current_time(time_str, sizeof(time_str));
218         printf("[%s] 创建哲学家线程%d\n", time_str, i + 1);
219     }
220
221     // 3. 等待所有哲学家线程结束
222     for (int i = 0; i < N; i++) {
223         pthread_join(phil_tids[i], NULL);
224     }
225
226     // 4. 释放资源
227     sem_destroy(&mutex);
228     for (int i = 0; i < N; i++) {
229         sem_destroy(&forks[i]);
230     }
231
232     time_t end_time = time(NULL);
233     double total_time = difftime(end_time, start_time);
234
235     printf("=====\n");
236     printf("实验结束: %d个哲学家共就餐%d次, 耗时%.2f秒\n", N, EAT_TIMES, total_time);
237     printf("就餐次数: \n");
238     for (int i = 0; i < N; i++) {
239         printf("哲学家%d: %d次\n", i + 1, eat_count[i]);
240     }
241     printf("=====\n");
242
243     return 0;
244 }
```

4、编译与运行

```
gcc dining_philosophers.c -o dining_philosophers -lpthread
```

```
./dining_philosophers > dining_log.txt 2>&1
```

```
LZH2311451@LZH2311451-pc:~$ vim dining_philosophers.c
LZH2311451@LZH2311451-pc:~$ gcc dining_philosophers.c -o dining_philosophers -lpthread
LZH2311451@LZH2311451-pc:~$ ./dining_philosophers > dining_log.txt 2>&1
LZH2311451@LZH2311451-pc:~$
```

将输出重定向到文件中，以便观察。

5、分析结果

首先来看最开始的部分输出，可以看到 5 个哲学家线程均能成功创建并首先进入思考状态。

每个哲学家在拿叉子（申请就餐）时，会直接进行一次检查，看看是否可以就餐，若可以就餐（左右叉子都空闲），则在退出临界区（申请结束）后，直接就餐。

如图中哲学家 1，其思考结束后，先进入临界区开始申请流程，将其状态改为饥饿，然后检查左右哲学家状态（check 操作没有消息打印，check 操作通过时会对执行 check 的哲学家的 forks 信号量进行 sem_post 操作，即允许该哲学家就餐），随后哲学家 1 退出临界区用 sem_wait 等待申请结果，由于哲学家 1 是第一个申请就餐的，所有叉子都空闲，所以它的 check 直接通过，也就可以就餐。

```
dining_log.txt X
=====
哲学家就餐实验：哲学家5个，共就餐10次
=====
[2026-01-01 22:22:58] 创建哲学家线程1
[2026-01-01 22:22:58] 哲学家1：线程启动
[2026-01-01 22:22:58] 哲学家1：开始思考，预计耗时3秒
[2026-01-01 22:22:58] 哲学家2：线程启动
[2026-01-01 22:22:58] 哲学家2：开始思考，预计耗时4秒
[2026-01-01 22:22:58] 创建哲学家线程2
[2026-01-01 22:22:58] 创建哲学家线程3
[2026-01-01 22:22:58] 哲学家3：线程启动
[2026-01-01 22:22:58] 创建哲学家线程4
[2026-01-01 22:22:58] 创建哲学家线程5
[2026-01-01 22:22:58] 哲学家4：线程启动
[2026-01-01 22:22:58] 哲学家5：线程启动
[2026-01-01 22:22:58] 哲学家5：开始思考，预计耗时4秒
[2026-01-01 22:22:58] 哲学家3：开始思考，预计耗时4秒
[2026-01-01 22:22:58] 哲学家4：开始思考，预计耗时5秒
[2026-01-01 22:23:01] 哲学家1：思考结束，准备就餐
[2026-01-01 22:23:01] 哲学家1：进入临界区（申请叉子）
[2026-01-01 22:23:01] 哲学家1：处于饥饿状态，等待左右叉子
[2026-01-01 22:23:01] 哲学家1：离开临界区（等待申请结果）
[2026-01-01 22:23:01] 哲学家1：开始就餐（第1次），预计耗时9秒
[2026-01-01 22:23:02] 哲学家2：思考结束，准备就餐
[2026-01-01 22:23:02] 哲学家2：进入临界区（申请叉子）
[2026-01-01 22:23:02] 哲学家2：处于饥饿状态，等待左右叉子
[2026-01-01 22:23:02] 哲学家2：离开临界区（等待申请结果）
[2026-01-01 22:23:02] 哲学家5：思考结束，准备就餐
[2026-01-01 22:23:02] 哲学家5：进入临界区（申请叉子）
```

随后哲学家 2 又思考结束，准备就餐，但是很明显，哲学家 1 正在就餐，与哲学家 2 的叉子冲突，因此哲学家 2 的首次 check 无法通过，于是在退出临界区并 sem_wait 等待申请结果后，就会进入阻塞状态，无法直接就餐，后续哲学家 1 就餐结束后，会对 check 其左右邻居，并唤醒可以就餐的哲学家。

如图，一段时间后，哲学家 1 就餐结束并 check 哲学家 2，但此时哲学家 3 正在就餐，check 不通过，随后哲学家 3 也就餐结束并 check 哲学家 2，此时哲学家 2 左右叉子都空闲，便被唤醒就餐。

```
[2026-01-01 22:23:09] 哲学家3：开始就餐（第2次），预计耗时6秒
[2026-01-01 22:23:10] 哲学家1：就餐结束
[2026-01-01 22:23:10] 哲学家1：进入临界区（释放叉子）
[2026-01-01 22:23:10] 哲学家1：就餐结束，释放左右叉子（总就餐次数：6/10）
[2026-01-01 22:23:10] 哲学家1：离开临界区（叉子已释放）
[2026-01-01 22:23:10] 哲学家1：开始思考，预计耗时5秒
[2026-01-01 22:23:10] 哲学家5：开始就餐（第1次），预计耗时7秒
[2026-01-01 22:23:14] 哲学家4：思考结束，准备就餐
[2026-01-01 22:23:14] 哲学家4：进入临界区（申请叉子）
[2026-01-01 22:23:14] 哲学家4：处于饥饿状态，等待左右叉子
[2026-01-01 22:23:14] 哲学家4：离开临界区（等待申请结果）
[2026-01-01 22:23:15] 哲学家1：思考结束，准备就餐
[2026-01-01 22:23:15] 哲学家1：进入临界区（申请叉子）
[2026-01-01 22:23:15] 哲学家1：处于饥饿状态，等待左右叉子
[2026-01-01 22:23:15] 哲学家1：离开临界区（等待申请结果）
[2026-01-01 22:23:15] 哲学家3：就餐结束
[2026-01-01 22:23:15] 哲学家3：进入临界区（释放叉子）
[2026-01-01 22:23:15] 哲学家3：就餐结束，释放左右叉子（总就餐次数：8/10）
[2026-01-01 22:23:15] 哲学家3：离开临界区（叉子已释放）
[2026-01-01 22:23:15] 哲学家3：开始思考，预计耗时4秒
[2026-01-01 22:23:15] 哲学家2：开始就餐（第1次），预计耗时3秒
[2026-01-01 22:23:17] 哲学家5：就餐结束
[2026-01-01 22:23:17] 哲学家5：进入临界区（释放叉子）
```

再来看最后的一部分输出，可以看到能够顺利的完成规定数量的任务，并且各线程能够正确的有序退出，无忙等待和死锁现象。

其中总就餐次数在每次拿叉子时就进行加 1，而不是在就餐结束后，否则会导致次数计算错误。


```
[2026-01-01 22:23:26] 哲学家4：进入临界区（申请叉子）
[2026-01-01 22:23:26] 哲学家4：总就餐次数已达到10次，停止就餐
[2026-01-01 22:23:26] 哲学家4：线程退出（个人就餐2次）
[2026-01-01 22:23:26] 哲学家3：就餐结束
[2026-01-01 22:23:26] 哲学家3：进入临界区（释放叉子）
[2026-01-01 22:23:26] 哲学家3：就餐结束，释放左右叉子（总就餐次数：10/10）
[2026-01-01 22:23:26] 哲学家3：离开临界区（叉子已释放）
[2026-01-01 22:23:26] 哲学家3：开始思考，预计耗时7秒
[2026-01-01 22:23:26] 哲学家1：思考结束，准备就餐
[2026-01-01 22:23:26] 哲学家1：进入临界区（申请叉子）
[2026-01-01 22:23:26] 哲学家1：总就餐次数已达到10次，停止就餐
[2026-01-01 22:23:26] 哲学家1：线程退出（个人就餐2次）
[2026-01-01 22:23:31] 哲学家5：就餐结束
[2026-01-01 22:23:31] 哲学家5：进入临界区（释放叉子）
[2026-01-01 22:23:31] 哲学家5：就餐结束，释放左右叉子（总就餐次数：10/10）
[2026-01-01 22:23:31] 哲学家5：离开临界区（叉子已释放）
[2026-01-01 22:23:31] 哲学家5：开始思考，预计耗时7秒
[2026-01-01 22:23:33] 哲学家3：思考结束，准备就餐
[2026-01-01 22:23:33] 哲学家3：进入临界区（申请叉子）
[2026-01-01 22:23:33] 哲学家3：总就餐次数已达到10次，停止就餐
[2026-01-01 22:23:33] 哲学家3：线程退出（个人就餐3次）
[2026-01-01 22:23:38] 哲学家5：思考结束，准备就餐
[2026-01-01 22:23:38] 哲学家5：进入临界区（申请叉子）
[2026-01-01 22:23:38] 哲学家5：总就餐次数已达到10次，停止就餐
[2026-01-01 22:23:38] 哲学家5：线程退出（个人就餐2次）
=====
实验结束：5个哲学家共就餐10次，耗时40.00秒
就餐次数：
哲学家1: 2次
哲学家2: 1次
哲学家3: 3次
哲学家4: 2次
哲学家5: 2次
=====
```

三、结论

本次实验采用 C 语言结合 POSIX 线程与信号量机制，成功实现了 5 个哲学家就餐问题。通过 check() 函数校验就餐条件、互斥信号量 mutex 保护共享状态、forks 信号量控制就餐权限，有效避免了死锁与数据竞争问题。实验结果表明，5 个哲学家线程能够有序交替进行思考与就餐，正确完成 10 次总就餐任务后正常退出，无忙等待或程序阻塞现象。